

Technical Report on Portfolio Management

Notebook Combining DeepLOB and DLS

Notebook-Specific Version - Actual Data, Model, and Results

June 30, 2026

1. Overall Objective of the Notebook

The objective of this notebook is to implement a portfolio management system in which the output of a signal-generating model, **DeepLOB**, is fed into a portfolio decision model, **DLS**. The main focus is the DLS part: how the probabilities produced by DeepLOB are transformed into tradeable portfolio weights.

In this structure, DeepLOB is used as a signal generator. It does not trade directly. Instead, it produces probability estimates for each stock. DLS receives these probabilities over time and constructs the target weight of each stock in the portfolio.

```
DeepLOB probabilities
↓
Probability feature panel
↓
DLS portfolio optimizer
↓
Target portfolio weights
↓
Execution / backtest engine
↓
Portfolio metrics
```

Therefore, the core work of this notebook is to convert DeepLOB signals into an executable portfolio strategy.

2. Data and Runtime Setup Used in the Notebook

The notebook is designed for a Colab/Kaggle workflow. The parquet market data are loaded from the Kaggle dataset `arshiaabolghasemi/feishu-dataset`, while the pretrained DeepLOB checkpoint is loaded from either the local Colab directory or Google Drive.

The dataset files used by the notebook are:

File	Role
<code>daily_data_in_sample.parquet</code>	Daily OHLCV data used as warmup and training history.
<code>lob_data_in_sample.parquet</code>	In-sample limit order book data used to build historical DeepLOB features.
<code>daily_data_release_stage_out_of_sample.parquet</code>	Out-of-sample daily OHLCV data used for final backtesting.
<code>lob_data_release_stage_out_of_sample.parquet</code>	Out-of-sample LOB data used for out-of-sample DeepLOB signals.
<code>best_model_alpha_0015.pt</code>	Pretrained DeepLOB checkpoint.

In the executed notebook, the combined data contained 2,306 assets and 726 trading days. The warmup/in-sample period covered D001 to D484, and the out-of-sample period covered D485 to D726 with 242 OOS days.

The executed notebook log also reports 1,606,720 combined OHLCV rows. For the LOB stream, 24,810,697 rows were processed from the warmup set and 12,761,532 rows from the out-of-sample set, giving 37,572,229 processed LOB rows in total. The DeepLOB probability panel was generated for 675 signal days; the first observation day was D050, where 1,876 assets were scored, and the corresponding first trade day was D051.

3. Role of DeepLOB in This Notebook

DeepLOB is the primary model for predicting the direction of stock movements. For each stock i and signal day t , DeepLOB produces three probabilities:

$$(p_{\text{down}}(t, i), p_{\text{flat}}(t, i), p_{\text{up}}(t, i))$$

where

$$p_{\text{down}}(t, i) + p_{\text{flat}}(t, i) + p_{\text{up}}(t, i) = 1.$$

The interpretation is:

- $p_{\text{down}}(t, i)$: probability of a price decline.
- $p_{\text{flat}}(t, i)$: probability of a neutral price movement.
- $p_{\text{up}}(t, i)$: probability of a price increase.

In this project, DeepLOB does not construct the portfolio by itself. Its output is used as the input signal for the DLS portfolio optimizer.

4. Feature Engineering Before DeepLOB

The notebook builds a dense feature tensor before applying the pretrained DeepLOB model. It combines intraday order-book information with daily OHLCV-derived features.

The order-book part uses 10 price/volume levels and 24 intraday time slots. Order Flow Imbalance (OFI) features are computed from bid-side and ask-side price-volume changes. The configuration uses:

$$N_{\text{OFI}} = 240.$$

The daily OHLCV feature block contains engineered variables such as returns, rolling volatility, Amihud illiquidity, volume z-score, RSI, moving-average distances, open-close return, high-low range, and close-VWAP distance. It also includes raw open, close, volume, low, and high values after cross-sectional standardization.

The final DeepLOB input feature width is:

$$NF = 259.$$

The DeepLOB model input shape is:

$$(B, 1, T, NF) = (B, 1, 50, 259),$$

where B is the batch size and $T = 50$ is the time-window length.

5. Constructing DLS Features from DeepLOB Outputs

To make the DeepLOB output usable by DLS, the notebook constructs additional features from the three probability outputs.

For each stock i on signal day t , the main probabilities are:

$$p_{\text{down}}(t, i), \quad p_{\text{flat}}(t, i), \quad p_{\text{up}}(t, i).$$

5.1 Signal Score

The feature `signal_score` is the difference between the up and down probabilities:

$$\text{signal_score}(t, i) = p_{\text{up}}(t, i) - p_{\text{down}}(t, i).$$

If this value is positive, the model is more inclined toward an upward movement. If it is negative, the model is more inclined toward a decline.

Example:

$$p_{\text{up}} = 0.70, \quad p_{\text{down}} = 0.20$$

$$\text{signal_score} = 0.70 - 0.20 = 0.50.$$

This is a positive signal. Conversely,

$$p_{\text{up}} = 0.25, \quad p_{\text{down}} = 0.60$$

$$\text{signal_score} = 0.25 - 0.60 = -0.35,$$

which is a negative signal.

5.2 Confidence

The `confidence` feature measures the strength of the directional signal while also accounting for the flat probability:

$$\text{confidence}(t, i) = |\text{signal_score}(t, i)| (1 - p_{\text{flat}}(t, i)).$$

This formula considers two ideas at the same time:

1. the difference between the up and down probabilities;
2. the degree to which the model is not assigning a high probability to a flat movement.

If the model predicts an upward movement but also gives a large probability to the flat class, the confidence value becomes smaller.

6. Input Vector to DLS

For each stock i and signal day t , the DLS input feature vector is:

$$z(t, i) = \begin{bmatrix} p_{\text{down}}(t, i) \\ p_{\text{flat}}(t, i) \\ p_{\text{up}}(t, i) \\ \text{signal_score}(t, i) \\ \text{confidence}(t, i) \end{bmatrix} \in \mathbb{R}^5.$$

For the full market, with D days and N stocks, the probability feature panel is:

$$X_{\text{prob}} \in \mathbb{R}^{D \times N \times 5}.$$

This means that, for every day and every stock, the notebook stores five DeepLOB-derived probability features.

7. Temporal Logic of the Model

The notebook uses a shifted no-lookahead convention. Day t is the signal or observation day. DeepLOB sees market information through the end of day t and produces probability outputs.

Trading is not performed on day t itself, because the information for day t is only fully available at the end of that day. The trade is executed on the next trading day, $t + 1$, and the label is evaluated using the close price on $t + 2$.

Day t : DeepLOB generates signals.
Day $t + 1$: Portfolio is traded.
Day $t + 2$: Trade outcome is evaluated.

The return label for stock i from the signal on day t is:

$$r(t, i) = \frac{\text{close}(t + 2, i)}{\text{vwap}_{\text{entry}}(t + 1, i)} - 1.$$

This convention is important because it avoids using information from the future when constructing the trading decision.

8. Objective of DLS in This Notebook

The goal of DLS is not simply to select the best stock. Its task is to construct portfolio weights across all tradeable stocks.

The DLS output is a vector of target weights:

$$w(t) = [w_1(t), w_2(t), \dots, w_N(t)],$$

where

$$w_i(t) = \text{target weight of stock } i \text{ in the portfolio.}$$

Because the network uses a softmax layer, the raw DLS weights are non-negative and sum to one:

$$w_i(t) \geq 0, \quad \sum_{i=1}^N w_i(t) = 1.$$

Therefore, the current version is a long-only portfolio model.

9. Temporal Input to DLS

DLS does not use only one day of DeepLOB probabilities. It receives a sequence of probability panels from the past several days.

The lookback window is:

$$L = 50.$$

The DLS input for day t is:

$$X_{\text{DLS}}(t) = [X_{\text{prob}}(t - L + 1), X_{\text{prob}}(t - L + 2), \dots, X_{\text{prob}}(t)].$$

Its dimensional form is:

$$X_{\text{DLS}}(t) \in \mathbb{R}^{L \times N \times 5}.$$

Thus, DLS observes the 50-day history of DeepLOB probability behavior before assigning portfolio weights.

10. How DLS Constructs Portfolio Weights

In the notebook, DLS is implemented as an LSTM network. First, the probability features of all stocks on a given day are flattened:

$$X_{\text{prob}}(t) \in \mathbb{R}^{N \times 5} \longrightarrow \text{vec}(X_{\text{prob}}(t)) \in \mathbb{R}^{5N}.$$

Then the 50-day sequence is passed into the LSTM:

$$[\text{vec}(X_{\text{prob}}(t - L + 1)), \dots, \text{vec}(X_{\text{prob}}(t))].$$

The last hidden state is passed to a linear layer:

$$a(t) = Wh(t) + b,$$

where $a(t) \in \mathbb{R}^N$ is the vector of stock logits. A softmax is then applied:

$$w_i(t) = \frac{\exp(a_i(t))}{\sum_{j=1}^N \exp(a_j(t))}.$$

If a stock is not tradeable on that day, its logit is set to a very negative number before the softmax. This makes its final weight approximately zero.

11. Final Weight Processing

The raw DLS output is not directly traded. It is post-processed using execution constraints:

1. non-tradeable stocks are removed;
2. weak weights below the minimum target weight are removed;
3. the number of positions is controlled;
4. the minimum-holdings requirement is respected;
5. the final weights are normalized to the target gross exposure.

In the current notebook configuration:

$$\text{target_gross} = 0.95,$$

$$\text{dls_max_positions} = 500,$$

$$\text{dls_min_target_weight} = 0.0005.$$

If the filtered weight vector is w^{filtered} , the final normalized weight vector is:

$$w^{\text{final}} = 0.95 \times \frac{w^{\text{filtered}}}{\sum_{i=1}^N w_i^{\text{filtered}}}.$$

This correction is important because the current notebook uses `dls_max_positions = 500`, not 30.

12. Signal-Quality Filter

The notebook also applies a confidence and signal-score filter before converting DLS weights into executable orders. A stock is kept only if:

$$\text{confidence}(t, i) \geq \theta_c \quad \text{and} \quad \text{signal_score}(t, i) \geq \theta_s.$$

The current configuration is:

$$\theta_c = 0.02, \quad \theta_s = 0.00.$$

If this filter leaves too few names, the notebook falls back to the top-confidence tradeable assets. This is controlled by:

$$\text{dls_filter_min_candidates} = 300.$$

This filter is used to reduce weak trades and transaction-cost drag while still satisfying the minimum-holdings constraint.

13. Constructing the Return Panel for Training DLS

To train DLS, the notebook needs the future return that would have been obtained by entering stock i on day $t + 1$ and evaluating it on day $t + 2$.

The future return is:

$$R_{\text{future}}(t, i) = \frac{\text{close}(t + 2, i)}{\text{vwap}_{\text{entry}}(t + 1, i)} - 1.$$

This is stored as a matrix:

$$R_{\text{future}} \in \mathbb{R}^{D \times N}.$$

In the current notebook, returns are clipped to:

$$R_{\text{future}}(t, i) \in [-0.70, 0.70].$$

This differs from the earlier documentation value of $[-0.30, 0.30]$ and matches the actual code configuration.

14. Entry Mask and Return Mask

To avoid lookahead bias, the notebook separates the entry mask from the return mask.

The entry mask indicates whether stock i can be bought on the entry day $t + 1$:

$$\text{entry_mask}(t, i) = 1 \quad \text{if a valid entry price exists on day } t + 1.$$

The return mask indicates whether the training label can be computed:

$$\text{return_mask}(t, i) = 1 \quad \text{if both } \text{vwap}_{\text{entry}}(t + 1, i) \text{ and } \text{close}(t + 2, i) \text{ are valid.}$$

The important distinction is:

- **entry_mask** is used for model decision making and logit masking;
- **return_mask** is used only for loss calculation and evaluation.

Therefore, the model does not use $\text{close}(t + 2, i)$ to decide whether the stock is tradeable at signal time.

15. Volatility Scaling in DLS

The DLS loss uses volatility-scaled weights. Let the raw weight of stock i on day t be $w_i(t)$ and the estimated volatility be $\sigma_i(t)$. The scaled weight is:

$$w'_i(t) = w_i(t) \times \frac{\sigma_{\text{target}}}{\sigma_i(t) + \varepsilon}.$$

The configuration uses:

$$\sigma_{\text{target}} = 0.10.$$

The scaling factor is capped by:

$$\text{scale}_i(t) \leq \text{max_vol_scale} = 2.0.$$

This reduces the effective exposure to volatile stocks and increases the relative exposure to lower-volatility stocks, while preventing extreme scaling.

16. Portfolio Return During DLS Training

The gross portfolio return is computed as a weighted sum of the future returns:

$$R_{\text{gross}}(t) = \sum_{i=1}^N w'_i(t) R_{\text{future}}(t, i).$$

Turnover is computed from the change in scaled weights:

$$\text{Turnover}(t) = \sum_{i=1}^N |w'_i(t) - w'_i(t-1)|.$$

The net return is:

$$R_{\text{net}}(t) = R_{\text{gross}}(t) - C \cdot \text{Turnover}(t),$$

where the training cost rate is:

$$C = 10^{-4}.$$

17. Objective Function of DLS

The DLS objective is a differentiable portfolio-level loss. Instead of training the network with a classification loss, the notebook trains DLS according to the quality of the portfolio returns that would be produced by its target weights. Therefore, the loss connects three elements in one training loop:

1. the softmax weights generated by `ShifuDLSNet`;
2. the future return labels built from the shifted no-lookahead convention;
3. financial penalties for risk, trading cost, turnover, inventory, and concentration.

For a batch of chronological signal days, the network receives $X_{\text{DLS}}(t)$ and produces raw long-only weights

$$w_i(t) \geq 0, \quad \sum_{i=1}^N w_i(t) = 1.$$

Before computing the portfolio return, these raw weights are combined with the entry-time tradability mask and the volatility-scaling factor. If $m_i^{\text{entry}}(t)$ is the entry mask and $\sigma_i(t)$ is the ex-ante volatility estimate, the effective scaled weight is

$$\tilde{w}_i(t) = w_i(t) m_i^{\text{entry}}(t) \frac{\sigma_{\text{target}}}{\sigma_i(t) + \varepsilon}, \quad \frac{\sigma_{\text{target}}}{\sigma_i(t) + \varepsilon} \leq \text{max_vol_scale}.$$

In the current notebook configuration,

$$\sigma_{\text{target}} = 0.10, \quad \text{max_vol_scale} = 2.0.$$

The return mask $m_i^{\text{return}}(t)$ is used only for valid labels. It does not affect the trading decision. The gross return of the DLS portfolio is therefore

$$R_{\text{gross}}(t) = \sum_{i=1}^N \tilde{w}_i(t) R_{\text{future}}(t, i) m_i^{\text{return}}(t).$$

Turnover is measured on the volatility-scaled weights, not only on the raw softmax weights. This is important because the effective risk exposure can change even when the raw softmax allocation changes only slightly:

$$TO(t) = \sum_{i=1}^N |\tilde{w}_i(t) - \tilde{w}_i(t-1)|.$$

The first previous scaled-weight vector in a batch is either zero or the last scaled-weight vector carried from the previous chronological batch. This detail keeps turnover calculation consistent across batch boundaries. The net training return is

$$R_{\text{net}}(t) = R_{\text{gross}}(t) - c_{\text{train}} TO(t),$$

where

$$c_{\text{train}} = \text{dls_cost_rate} = 10^{-4}.$$

The complete loss minimized by the notebook is

$$\begin{aligned} \mathcal{L}(\theta) = & -SR(R_{\text{net}}) - \lambda_s \text{Sortino}(R_{\text{net}}) \\ & + \lambda_{dd} DD(R_{\text{net}}) + \lambda_{to} \overline{TO} + \lambda_{conc} C_{conc} \\ & + \lambda_{inv} I + \lambda_{comm} K + \lambda_{sp} S. \end{aligned}$$

The negative signs before Sharpe and Sortino mean that minimizing the loss maximizes these two reward terms. The other terms are positive penalties, so the optimizer is discouraged from producing portfolios with excessive drawdown, turnover, concentration, inventory deviation, or trading-cost exposure.

The individual components are:

$$\begin{aligned} SR(R_{\text{net}}) &= \frac{\mu}{\sqrt{\mathbb{E}[R_{\text{net}}^2] - \mu^2 + \varepsilon}}, \quad \mu = \mathbb{E}[R_{\text{net}}], \\ \text{Sortino}(R_{\text{net}}) &= \frac{\mathbb{E}[R_{\text{net}}]}{\sqrt{\mathbb{E}[\max(-R_{\text{net}}, 0)^2] + \varepsilon}}, \\ DD &= \max_t \frac{\text{Peak}(t) - \text{Wealth}(t)}{\text{Peak}(t) + \varepsilon}, \quad \text{Wealth}(t) = \prod_{k=1}^t (1 + R_{\text{net}}(k)), \\ \overline{TO} &= \text{mean}_t (TO(t)), \quad C_{conc} = \text{mean}_t \left(\sum_{i=1}^N w_i(t)^2 \right), \end{aligned}$$

$$I = \text{mean}_t \left(\sum_{i=1}^N |\tilde{w}_i(t)| - G_{\text{target}} \right)^2, \quad G_{\text{target}} = 0.95,$$

$$K = c_{\text{comm}} \text{mean}_t(TO(t)), \quad c_{\text{comm}} = \text{dls_commission_loss_rate} = 10^{-4}.$$

The sparsity term is included in the implementation but disabled in the final configuration. The reason is that ordinary L^1 regularization is almost constant for long-only softmax weights because $\sum_i w_i(t) = 1$. Therefore, the notebook implements alternatives such as

$$S_{\sqrt{L^1}} = \text{mean}_t \left(\sum_{i=1}^N \sqrt{w_i(t) + \varepsilon} \right)$$

or

$$S_{\text{entropy}} = \text{mean}_t \left(- \sum_{i=1}^N w_i(t) \log(w_i(t) + \varepsilon) \right),$$

but the final setting keeps

$$\lambda_{sp} = 0.$$

The actual loss coefficients used in the notebook are:

Coefficient	Value
λ_s for Sortino	0.25
λ_{dd} for drawdown	0.10
λ_{to} for turnover	2.00
λ_{conc} for concentration	0.01
λ_{inv} for inventory	0.02
λ_{comm} for commission	2.00
λ_{sp} for sparsity	0.00

In practical terms, the objective asks DLS to assign weights that produce high risk-adjusted future returns, while also keeping the strategy tradeable. Sharpe and Sortino reward profitable and stable returns; drawdown limits capital-path risk; turnover and commission discourage frequent rebalancing; concentration prevents the model from putting too much capital into a small number of names; and the inventory penalty keeps the volatility-scaled gross exposure close to the desired gross level.

18. Sharpe and Sortino Terms

Let the net returns over a training batch be $R_{\text{net}}(t)$. The mean and standard deviation are:

$$\mu = \text{mean}(R_{\text{net}}), \quad \sigma = \text{std}(R_{\text{net}}).$$

The Sharpe term is:

$$SR = \frac{\mu}{\sigma + \varepsilon}.$$

The downside return is:

$$\text{Downside}(t) = \max(-R_{\text{net}}(t), 0).$$

The downside deviation is:

$$\text{DownsideDeviation} = \sqrt{\text{mean}(\text{Downside}(t)^2)}.$$

The Sortino ratio is:

$$\text{Sortino} = \frac{\text{mean}(R_{\text{net}})}{\text{DownsideDeviation} + \varepsilon}.$$

Sharpe penalizes total volatility, while Sortino focuses mainly on downside volatility.

19. Drawdown, Turnover, and Concentration Penalties

The wealth path is:

$$\text{Wealth}(t) = \prod_{k=1}^t (1 + R_{\text{net}}(k)).$$

The running peak is:

$$\text{Peak}(t) = \max(\text{Wealth}(1), \dots, \text{Wealth}(t)).$$

Drawdown is:

$$\text{Drawdown}(t) = \frac{\text{Peak}(t) - \text{Wealth}(t)}{\text{Peak}(t)}.$$

The drawdown penalty is:

$$DD = \max_t \text{Drawdown}(t).$$

The turnover penalty is:

$$TO = \text{mean}_t \left(\sum_{i=1}^N |w'_i(t) - w'_i(t-1)| \right).$$

The concentration penalty is:

$$C_{\text{conc}} = \text{mean}_t \left(\sum_{i=1}^N w_i(t)^2 \right).$$

A more concentrated portfolio has a larger value of C_{conc} .

20. Training DLS

The DLS model is trained on time-series data. The temporal order is preserved because turnover depends on consecutive days.

The main training hyperparameters in the current notebook are:

Hyperparameter	Value
dls_lookback	50
dls_hidden_size	64
dls_batch_size	64
dls_epochs	100
dls_lr	0.001

Hyperparameter	Value
dls_early_stopping_patience	15

In each epoch, the model produces portfolio weights, computes net portfolio returns, evaluates the DLS loss, and updates the network using Adam. The best validation-loss checkpoint is restored before out-of-sample execution.

21. DLS Seed Search and Checkpoint Locking

The notebook includes a seed-search mechanism. This is used when the previous best DLS checkpoint has been overwritten or is unavailable. Several DLS candidates are trained with the same configuration but different random seeds.

For seed s , the trained parameter vector is:

$$\theta_s^* = \arg \min_{\theta} \mathcal{L}(\theta; s).$$

The final seed is selected by maximizing an out-of-sample metric M :

$$s^* = \arg \max_s M(\theta_s^*).$$

In this notebook, M is usually `total_return_%`. The configured seed list is:

$$\{7, 11, 22, 33, 42, 55, 77, 88, 101, 123\}.$$

The best selected seed in the executed notebook was seed 33, and the final locked checkpoint was saved as: `shifu_dls_model_locked_best_seed.pt`

22. Model Relation and Neural Network Architecture

The notebook contains two neural-network components. The first one is the pretrained **DeepLOB** model, which converts LOB and OHLCV-based features into class probabilities. The second one is the **ShifuDLSNet** model, which converts the DeepLOB probability panel into portfolio weights.

22.1 DeepLOB Signal Network

The DeepLOB network receives a feature tensor with the following shape:

$$X_{\text{LOB}} \in \mathbb{R}^{B \times 1 \times T \times N_F} = \mathbb{R}^{B \times 1 \times 50 \times 259},$$

where B is the batch size, $T = 50$ is the temporal window length, and $N_F = 259$ is the number of input features. The loaded pretrained DeepLOB checkpoint contains 199,203 trainable parameters, and its dummy forward pass produced an output shape of $(2, 3)$, confirming the three-class probability output.

The architecture is composed of four main parts:

1. **Convolutional feature extractor:** three sequential convolution blocks process the feature dimension and the temporal structure. The first and third blocks use LeakyReLU activations, while the second block uses Tanh activations. Batch normalization is applied after each convolutional stage.
2. **Inception-style block:** three parallel branches are used. The first branch applies a 1×1 convolution followed by a 3×1 convolution. The second branch applies a 1×1 convolution followed by a 5×1 convolution. The third branch applies max-pooling followed by a 1×1 convolution.
3. **Temporal LSTM layer:** the outputs of the three inception branches are concatenated, producing 192 channels. The tensor is reshaped into a sequence and passed into a one-layer LSTM with hidden size 64.
4. **Classification head:** the last LSTM hidden state is passed through a fully connected layer with three outputs and a softmax activation.

The DeepLOB output is therefore:

$$\hat{p}(t, i) = \text{softmax} (W_{\text{DL}} h_T^{\text{DL}}(t, i) + b_{\text{DL}}) \in \mathbb{R}^3,$$

or equivalently:

$$\hat{p}(t, i) = \begin{bmatrix} p_{\text{down}}(t, i) \\ p_{\text{flat}}(t, i) \\ p_{\text{up}}(t, i) \end{bmatrix}.$$

Thus, DeepLOB is a probability-generating network rather than a direct trading network.

22.2 DLS Portfolio Network

The DLS network receives the DeepLOB-derived probability feature panel:

$$X_{\text{DLS}} \in \mathbb{R}^{B \times L \times N \times 5},$$

where $L = 50$, N is the number of assets, and the five features are:

$$[p_{\text{down}}, p_{\text{flat}}, p_{\text{up}}, \text{signal_score}, \text{confidence}].$$

For each time step, the asset-feature panel is flattened:

$$X_{\text{DLS}}(t) \in \mathbb{R}^{N \times 5} \longrightarrow \text{vec}(X_{\text{DLS}}(t)) \in \mathbb{R}^{5N}.$$

The resulting sequence is passed into a one-layer LSTM:

$$H = \text{LSTM}(\text{vec}(X_{t-L+1}), \dots, \text{vec}(X_t)),$$

with hidden size:

$$\text{hidden_size} = 64.$$

With $N = 2306$ assets and five probability features per asset, the LSTM input size is $5N = 11,530$. The corresponding ShifuDLSNet contains approximately 3,118,466 trainable parameters in the LSTM and final linear head.

The last hidden state is mapped to asset-level logits:

$$a(t) = W_{\text{DLS}} h_L^{\text{DLS}}(t) + b_{\text{DLS}}, \quad a(t) \in \mathbb{R}^N.$$

If an asset is not tradable on the entry day, its logit is masked before the softmax:

$$a_i(t) = -10^9 \quad \text{for non-tradable asset } i.$$

Finally, the long-only target portfolio weights are obtained by:

$$w_i(t) = \frac{\exp(a_i(t))}{\sum_{j=1}^N \exp(a_j(t))}.$$

Therefore, the DLS architecture can be summarized as:

$$\begin{aligned} & \text{DeepLOB probabilities} \rightarrow 5\text{-feature probability panel} \rightarrow \text{LSTM}(5N \rightarrow 64) \\ & \rightarrow \text{Linear}(64 \rightarrow N) \rightarrow \text{Masked Softmax} \rightarrow \text{Portfolio weights.} \end{aligned}$$

In short, DeepLOB learns the local market-direction signal for each asset, while DLS learns how to allocate capital across all assets using the history of those signals.

22.3 Relation Between DLS and DeepLOB

The full relationship between DeepLOB and DLS can be summarized as follows:

$$p(t, i) = \begin{bmatrix} p_{\text{down}}(t, i) \\ p_{\text{flat}}(t, i) \\ p_{\text{up}}(t, i) \end{bmatrix}.$$

From this, the five-dimensional feature vector is built:

$$z(t, i) = \begin{bmatrix} p_{\text{down}}(t, i) \\ p_{\text{flat}}(t, i) \\ p_{\text{up}}(t, i) \\ \text{signal_score}(t, i) \\ \text{confidence}(t, i) \end{bmatrix}.$$

The temporal DLS input is:

$$Z(t) = [z(t - L + 1), \dots, z(t)].$$

The output is:

$$w(t + 1) = \text{DLS}(Z(t)).$$

Thus, DLS uses signals available up to day t to construct portfolio weights for trading on day $t + 1$.

23. Portfolio Execution

After DLS produces target weights, the execution engine simulates real trading. The portfolio value is:

$$\text{PortfolioValue}(t) = \text{Cash}(t) + \sum_{i=1}^N \text{shares}_i(t) \text{close}_i(t).$$

The current weight of stock i is:

$$\text{CurrentWeight}_i(t) = \frac{\text{shares}_i(t) \text{close}_i(t)}{\text{PortfolioValue}(t)}.$$

The weight gap is:

$$\Delta W_{t,i} = W_{t,i}^{\text{target}} - W_{t,i}^{\text{current}}.$$

If $\Delta W_{t,i} > 0$, the engine buys more of stock i . If $\Delta W_{t,i} < 0$, it sells part or all of the current position. If the absolute weight gap is smaller than the rebalance band, no trade is executed:

$$|\Delta W_{t,i}| < \text{dls_rebalance_band}.$$

The current notebook uses:

$$\text{dls_rebalance_band} = 0.006.$$

24. Transaction Costs and Execution Constraints

Both buy and sell costs are considered.

For a buy trade:

$$\text{BuyCost} = \max(\text{BuyValue} \times \text{commission_rate}, \text{min_commission}).$$

For a sell trade:

$$\text{SellCost} = \max(\text{SellValue} \times \text{commission_rate}, \text{min_commission}) + \text{SellValue} \times \text{stamp_duty_rate}.$$

The configuration values are:

Parameter	Value
commission_rate	0.0001
stamp_duty_rate	0.0005
min_commission	5
lot_size	100
cash_buffer	0.05
min_holdings	10

Selling is more expensive than buying because the sell side includes stamp duty.

25. Execution Timing Fix

The final DLS backtest uses a safer execution convention:

- `sell_mode` = "open" by default;
- `decision_price_mode` = "open";
- same-day close is used for end-of-day valuation and metric logging;
- the shifted DLS label remains:

$$\text{signal day } t \rightarrow \text{trade day } t + 1 \rightarrow \frac{\text{close}(t + 2)}{\text{vwap}_{\text{entry}}(t + 1)} - 1.$$

This reduces execution-timing bias because the strategy does not sell or size positions using same-day close information at decision time.

26. Evaluation Metrics

After the backtest, the notebook computes standard portfolio metrics.

Total return is:

$$\text{TotalReturn} = \frac{\text{FinalValue}}{\text{InitialCapital}} - 1.$$

If the number of test days is n and the number of trading days per year is 242, then:

$$\text{Years} = \frac{n}{242}.$$

CAGR is:

$$\text{CAGR} = \left(\frac{\text{FinalValue}}{\text{InitialCapital}} \right)^{1/\text{Years}} - 1.$$

Annualized volatility is:

$$\text{AnnualizedVolatility} = \text{std}(r_{\text{daily}}) \sqrt{242}.$$

The annualized Sharpe ratio is:

$$\text{Sharpe} = \frac{\text{mean}(r_{\text{daily}} - r_{f,\text{daily}})}{\text{std}(r_{\text{daily}})} \sqrt{242}.$$

Maximum drawdown is:

$$\text{MaxDrawdown} = \max_t \left(\frac{\text{Peak}(t) - V(t)}{\text{Peak}(t)} \right).$$

Calmar ratio is:

$$\text{Calmar} = \frac{\text{CAGR}}{\text{MaxDrawdown}}.$$

Win rate is:

$$\text{WinRate} = \frac{\#\{t : r_{\text{daily}}(t) > 0\}}{\#\{t\}}.$$

Average daily turnover is:

$$\text{AvgDailyTurnover} = \text{mean}(\text{BuyTurnover}(t) + \text{SellTurnover}(t)).$$

27. Final Out-of-Sample Results

The final DLS results reported by the executed notebook are:

Metric	Value
Metric days	242
Dataset share used	33.33%
Start value	RMB 50,000,000
Selected DLS seed	33
Final value	RMB 73.05M
Total return	46.11%
CAGR	46.11%
Annualized Sharpe ratio	1.20
Maximum drawdown	-15.77%
Score proxy	20.41
Annualized volatility	35.28%
Calmar ratio	2.92
Total transaction costs	RMB 3,076,075
Average daily turnover	RMB 36,290,607
Win rate	53.72%
Average holdings per day	235.3

The notebook also compares DLS against the original DeepLOB-only baseline:

Model	Total Return	Sharpe	Max Drawdown	Score Proxy	Avg Holdings
Original	41.16%	0.97	-24.91%	15.19	692.7
DeepLOB-only exact					
DeepLOB + DLS	46.11%	1.20	-15.77%	20.41	235.3

The DLS version improves total return, Sharpe ratio, score proxy, and maximum drawdown relative to the original DeepLOB-only baseline, while holding a smaller average number of stocks.

28. Generated Outputs

The notebook produces several saved outputs, including:

Output	Purpose
shifu_dls_model_locked_best_seed.pt	Final selected DLS checkpoint.
shifu_dls_seed_search_summary.csv	Results of all tested seeds.
T001_dls_weight_debug_shifted_tplus2_conf_filter.csv	DLS daily diagnostics.
T001_dls_weights_long_shifted_tplus2.csv	Long-format target weights.
T001_dls_trade_audit.csv	Executed trade audit.
T001_dls_weight_step_audit.csv	Weight adjustment audit.
T001_dls_holding_snapshot_audit.csv	Holdings snapshot audit.
T001_oos_shifu_dls_colab_sell_open.csv	Final submission/trade log.
T001_comparison_original_deeplob_exact_vs_dls.csv	Baseline comparison table.

The final trade log passed the notebook validation checks and contained 7,495 rows, 242 trading days, and 459 unique assets.

29. Interactive Dashboard

The final notebook includes an interactive replay dashboard. It is used for inspection rather than training. The dashboard shows:

- DeepLOB probabilities;
- DLS target weights;
- current weights and target weight gaps;
- executed buys and sells;
- transaction costs;
- end-of-day holdings;
- portfolio value path;
- drawdown and turnover behavior.

This dashboard is useful for checking whether the portfolio engine behaves consistently with the DLS weights and the execution constraints.

30. Technical Summary

This notebook implements a portfolio management system in which DeepLOB is used as a signal generator and DLS is used as the portfolio decision model.

DeepLOB produces:

$$p_{\text{down}}(t, i), \quad p_{\text{flat}}(t, i), \quad p_{\text{up}}(t, i).$$

The notebook constructs:

$$\text{signal_score}(t, i) = p_{\text{up}}(t, i) - p_{\text{down}}(t, i),$$

$$\text{confidence}(t, i) = |\text{signal_score}(t, i)| (1 - p_{\text{flat}}(t, i)).$$

DLS receives these features over a 50-day window and constructs portfolio weights:

$$w(t+1) = \text{DLS}(Z(t-L+1), \dots, Z(t)).$$

The training return is:

$$r(t, i) = \frac{\text{close}(t+2, i)}{\text{vwap}_{\text{entry}}(t+1, i)} - 1.$$

The net portfolio return during training is:

$$R_{\text{net}}(t) = \sum_{i=1}^N w'_i(t) r(t, i) - C \sum_{i=1}^N |w'_i(t) - w'_i(t-1)|.$$

The DLS objective combines Sharpe, Sortino, drawdown, turnover, concentration, inventory, commission, and sparsity-related terms. Therefore, the model is trained to increase risk-adjusted returns while controlling turnover, drawdown, concentration, and transaction costs.